Course: Compiler Optimization.

Assignment 3: Part 1

Shreyas Udaya (pid: shreyasudaya)

Sanjula Thiranjaya (pid: sanjulatb)

5.1 Dominators

The dominators pass is used later in LICM and is a crucial component of LICM.

A node of a CFG (basic block) dominates a node if every possible path from the entry node must pass through.

In the implemented function, a Dominator_Tree is implemented to store the dominance information. So that it can be used by the LICM pass.

The *Dominator_Tree* has two helper functions

- dominates – check if A dominates B using the IDoms map
- getIDom - to get the immediate dominator of a block

Analysis

Microbenchmark 1: tests\dominators\test1.c

Example from the assignment

```
int main() {
    int result;
    int n=1;
    while(n++ < 100) {
        result = 0;
        result+=2;
        result+=3;
        result+=4;
        result/=2;
    }
    return result;
}
```

Matches the expected result:

```
opt -load-pass-plugin=build/unifiedpass.so -passes="dominators" build/test
test1-opt.bc
Dominator Tree for main
Block %entry -> IDom: None (Root)
Block %while.cond -> IDom: %entry
Block %while.body -> IDom: %while.cond
Block %while.end -> IDom: %while.cond
```

Microbenchmark 2: tests\dominators\test2.c

This test is a **"Diamond inside a Diamond."** It pushes our dominators pass to its limits.

```
int test2(int x) {
    int a = 0;
    int b = 1;
    if (x > 0) {
        a = a + 2;
        if (x % 2 == 0) {
            b = b * 3;
        } else {
            b = b + 5;
        }
    } else {
        a = a - 1;
    }
    int c = a + b;
    return c;
}
```

The result matches the observation from the CFG

```
opt -load-pass-plugin=build/unifiedpass.so -passes='dominators' build/tests/d
test2-opt.bc
Dominator Tree for test2
Block %entry -> IDom: None (Root)
Block %if.then -> IDom: %entry
Block %if.then2 -> IDom: %if.then
Block %if.else -> IDom: %if.then
Block %if.end -> IDom: %if.then
Block %if.else4 -> IDom: %entry
Block %if.end5 -> IDom: %entry
```

Microbenchmark 3: tests\dominators\test3.c

Testing of the pass handles blocks: Handled CORRECTLY.

```
int test3(int n) {
    int sum = 0;
    int i = 0;
    while (i < n) {
        if (i % 3 == 0) {
            sum += i;
        } else {
            sum += 2 * i;
        }
```

```
        i++;
    }
    // Unreachable block
    if (0) {
        sum = -999;
    }
    return sum;
```

The result matches the observation from the CFG

```
opt -load-pass-plugin=build/unifiedpass.so -passes='dominators' build/tests
test3-opt.bc
Dominator Tree for test3
Block %entry -> IDom: None (Root)
Block %while.cond -> IDom: %entry
Block %while.body -> IDom: %while.cond
Block %if.then -> IDom: %while.body
Block %if.else -> IDom: %while.body
Block %if.end -> IDom: %while.body
Block %while.end -> IDom: %while.cond
```

5.2 Dead Code Elimination

- Algorithm listed in Figure 3 of the paper: "Complexity of Data Flow Analysis for Non-Separable Frameworks", B. Karkare and U. Khedker, arXiv, October 2006, which is also shown in Figure in the assignment is used for this pass.
- isLive function given in the assignment is used for this pass.
- For each basic block, the following sets are generated, and the transfer function is applied. This runs until convergence
    - FaintOut
    - FaintIn
    - FaintGen
    - FaintKill
- Important distinction is unlike the other data-flow analysis problems, faint analysis is a non-seperable problem. Hence the FaintKill set depends on FaintOut set.
- That's why we need to recalculate all 4 sets in each iteration.
- FaintKill set defined in the paper is similar to what was discussed in the class. It is the union of ConstantKill and DependentKill sets.

Analysis

Microbenchmark 1: tests\dead-code-elimination\test1.c

This is the example from the assignment. The pass is able to successfully do the dead code elimination.

```
=== main ===
Instructions that can be deleted:
  %add1 = add nsw i32 %add, 96
  %sub = sub nsw i32 %add, 50
  %mul = mul nsw i32 %add, 96
  %add2 = add nsw i32 %add, 50
  %mul3 = mul nsw i32 %add, 96
  %c.0 = phi i32 [ %sub, %if.then ], [ %add2, %if.else ]
  %add4 = add nsw i32 -46, %c.0
Removing Instruction:   %add1 = add nsw i32 %add, 96
Removing Instruction:   %sub = sub nsw i32 %add, 50
Removing Instruction:   %mul = mul nsw i32 %add, 96
Removing Instruction:   %add2 = add nsw i32 %add, 50
Removing Instruction:   %mul3 = mul nsw i32 %add, 96
Removing Instruction:   %c.0 = phi i32 [ undef, %if.then ], [ undef, %if.else ]
Removing Instruction:   %add4 = add nsw i32 -46, undef
```

Microbenchmark 2: tests\dead-code-elimination\test2.c

This microbenchmark has dead chains. Once instruction becomes dead will make the other instructions which were live due to that instruction dead. The dead variables are commented on in the test2.c and the result after running the pass is as below.

```
=== main ===
Instructions that can be deleted:
  %mul = mul nsw i32 %add, 2
  %add1 = add nsw i32 %mul, 7
  %sub = sub nsw i32 %add1, 1
  %mul2 = mul nsw i32 %sub, %sub
  %add4 = add nsw i32 %add1, %sub3
  %sub6 = sub nsw i32 %sub, %add5
  %mul7 = mul nsw i32 %p.0, 3
  %add8 = add nsw i32 %mul7, 3
  %mul9 = mul nsw i32 %add8, %add8
Removing Instruction:   %mul = mul nsw i32 %add, 2
Removing Instruction:   %add1 = add nsw i32 undef, 7
Removing Instruction:   %sub = sub nsw i32 undef, 1
Removing Instruction:   %mul2 = mul nsw i32 undef, undef
Removing Instruction:   %add4 = add nsw i32 undef, %sub3
Removing Instruction:   %sub6 = sub nsw i32 undef, %add5
Removing Instruction:   %mul7 = mul nsw i32 %p.0, 3
Removing Instruction:   %add8 = add nsw i32 undef, 3
Removing Instruction:   %mul9 = mul nsw i32 undef, undef
```

Microbenchmark 3: tests\dead-code-elimination\test3.c

This microbenchmark will try to hit the corner cases to validate the DCE pass. Cases tested here are,

- Is function calls(*mayHaveSideEffects*) with side-effects preserved although the variable that uses the value returned from it is dead
- Is *DbgInfoIntrinsic* preserved
- Are stores preserved

From the results, it can be confirmed that the implemented DCE pass works correctly for these cases.

```
=== sideEffectFunc ===
Instructions that can be deleted:
=== main ===
Instructions that can be deleted:
  %mul3 = mul nsw i32 %add, 3
  %add4 = add nsw i32 %mul3, %call
  %sub = sub nsw i32 %add4, %0
  %mul5 = mul nsw i32 %sub, %sub
Removing Instruction:   %mul3 = mul nsw i32 %add, 3
Removing Instruction:   %add4 = add nsw i32 undef, %call
Removing Instruction:   %sub = sub nsw i32 undef, %0
Removing Instruction:   %mul5 = mul nsw i32 undef, undef
```

Following is the percentage of dynamic instructions saved due to this optimization.

```
root@a39b751d1f03:/work# ./lli-compare.sh ./build/tests/dead-code-elimination/
=========================================================================
Bitcode Pair          | m2r        | opt        | Diff (%)
=========================================================================
test1                 | 9          | 5          |     4 (+44.44%)
test2                 | 16         | 8          |     8 (+50.00%)
test3                 | 23         | 19         |     4 (+17.39%)
=========================================================================
```

5.3 Loop Invariant Code Motion

- The pass follows the guidelines given in the assignment. It's assumed that the input to code to this function is already transformed with m2r and loop-simplify passes.
- It will ensure that all loops have a preheader, a header and only one back edge.
- Hence the pass is done assuming the above.
- The test code is passed through m2r and loop-simplify in the makefile before applying loop-invariant-code-motion pass.

Invariance Check:

- The following checks are performed to remove the instructions that might not be safe to hoist. If one of these is hit, it is marked as variant
    - I.getType()->isVoidTy()
    - I.isTerminator() isa<PHINode>(&I))
    - I.mayReadFromMemory()
    - I.mayHaveSideEffects()
    - !isSafeToSpeculativelyExecute(&I))
- Then if the operands of the instruction are checked
    - If the operands are constant or function arguments, the instruction can be marked invariant.
    - If the operands are resulting from another instruction, it is checked whether the instruction is inside the loop or not.
        - For this check, reaching definitions is important
        - **But as the IR is in SSA form, values are not redefined. Hence, we can get the basic block that generates the value and check whether the basic block is inside the loop or not.**
        - If the reaching definitions are from outside the loop, the instruction can be marked as invariant

Dominance Check:

For the instruction to be safe for code motion, the instruction should dominate all loop exits. This can be determined by the dominance tree calculated using our dominators pass. Following are the steps,

- Build the dominance tree of the function
- The helper function in the Dominator_Tree class will be used to calculate the dominance of two basic blocks
- Using the loop information, we can extract the exiting blocks, then we can check whether the basic block with the instruction dominates all exists.

When one instruction is marked as invariant and hoisted, the instructions that use the hoisted instruction can become invariant due to the hoisting. Hence we maintain a hoisted set and a worklist and employ an iterative algorithm to catch all the transitive invariance in the loops.

After convergence, the instructions are moved to the preheader using the hoisted set.

As described in the assignment, when doing this without *loop-rotate*, the invariant instructions will not dominate the loop exits. Hence, we need to introduce a landing pad. Then transform the cfg as shown in assignment(This is basically what is done by loop-rotate inbuilt pass).

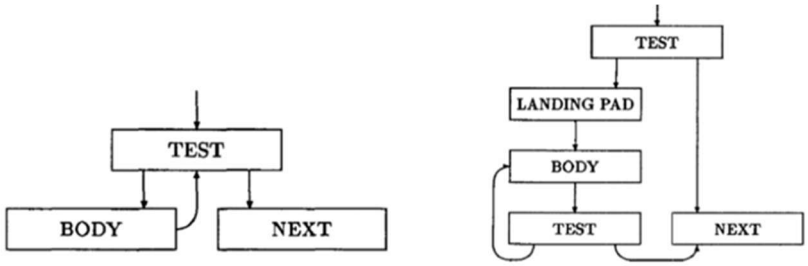
As using loop-rotate is restricted, we have implemented a function to so this transformation.

rotateLoop

- Two new basic blocks are created.
    - loop.landingpad
    - loop.cond
- Loop Body will be the successor to Landing Pad
- Loop Cond will be the successor of the Latch (Initial Basic Block with back edge)
- Loop Cond will have the cloned condition from the original preheader, and it will have a back edge to the Loop Body
- Original loop header (TEST in the figure) will be placed before the landing pad.
- *Basic analogy is the conversion of a while loop to a conditional do-while.*

PHI Nodes

- Doing this naively will result in LLVM verifier errors.
- That's due to not updating the PHI nodes in the basic blocks.
- In the figure,
    - NEXT block, which had only one predecessor, now has two
    - TEST block that had two predecessors, now has one
    - BODY block that had one predecessor, now has two
- Hence, we have to update all the PHI nodes with corresponding value from each path to preserve the semantic correctness after rotating the loop.
- This is also implemented inside the rotateLoop function.



Doing loop rotate before checking dominance will show that some of the invariant instructions can be safely hoisted to the new landing pad.

Analysis

Microbenchmark 1: tests\loop-invariant-code-motion\test1.c

This is the code from the assignment. Running the pass on this will mark and hoist the invariant instruction correctly.

```
opt -load-pass-plugin=build/unifiedpass.so -passes='loop-invariant-code-motion' build/te
2r.bc -o build/tests/loop-invariant-code-motion/test1-opt.bc

--- Analyzing Rotated Loop: for.body ---
    [SAFE - MARKED FOR HOISTING]   %mul = mul nsw i32 %x, 2
--- Successfully hoisted 1 instructions. ---
```

The IR will be slightly different from the given IR in the assignment due to the addition of landing pad and loop rotation. But it would have similar reduction of dynamic instruction count. Summary of dynamic instruction count is at the end of analysis.

Following is the optimized IR:

```
define dso_local i32 @test(i32 noundef %x) #0 {
entry:
  br label %for.cond

loop.landingpad:                                  ; preds = %for.cond
  %mul = mul nsw i32 %x, 2   //HOISTED INSTRUCTION TO LANDING PAD
  br label %for.body

for.cond:                                         ; preds = %entry
  %p.0 = phi i32 [ 0, %entry ] //LOOP CONDITION BEFORE LANDING PAD
  %i.0 = phi i32 [ 0, %entry ]
  %cmp = icmp slt i32 %i.0, 100
  br i1 %cmp, label %loop.landingpad, label %for.end

for.body:                                     ; preds = %loop.cond, %loop.landingpad
  %p.0.rot = phi i32 [ 0, %loop.landingpad ], [ %mul, %loop.cond ]
  %i.0.rot = phi i32 [ 0, %loop.landingpad ], [ %inc, %loop.cond ]
  br label %for.inc

for.inc:                                          ; preds = %for.body
  %inc = add nsw i32 %i.0.rot, 1
  br label %loop.cond, !llvm.loop !6
```

```
loop.cond:                                          ; preds = %for.inc
  %0 = icmp slt i32 %inc, 100
  br i1 %0, label %for.body, label %for.end  //LOOP CONDITION AFTER LOOP LATCH

for.end:                                        ; preds = %loop.cond, %for.cond
  %p.0.lcssa = phi i32 [ 0, %for.cond ], [ %p.0.rot, %loop.cond ]
  ret i32 %p.0.lcssa
}
```

Microbenchmark 2: tests\loop-invariant-code-motion\test2.c

Testcase with nested loops and invariants in different nested levels of the loops. The pass can handle nested loops and identify the invariants and place the invariant instructions at the correct landing pad.

```
opt -load-pass-plugin=build/unifiedpass.so -passes='loop-invariant-code-motion' build/tests/loop
2r.bc -o build/tests/loop-invariant-code-motion/test2-opt.bc

--- Analyzing Rotated Loop: for.body9 ---
    [SAFE - MARKED FOR HOISTING]   %add11 = add nsw i32 10, 5
    [SAFE - MARKED FOR HOISTING]   %add12 = add nsw i32 %add11, %mul
    [SAFE - MARKED FOR HOISTING]   %add10 = add nsw i32 %add4, 7
--- Successfully hoisted 3 instructions. ---

--- Analyzing Rotated Loop: for.body3 ---
    [SAFE - MARKED FOR HOISTING]   %mul5 = mul nsw i32 10, 5
    [SAFE - MARKED FOR HOISTING]   %add4 = add nsw i32 %mul, 3
--- Successfully hoisted 2 instructions. ---

--- Analyzing Rotated Loop: for.body ---
    [SAFE - MARKED FOR HOISTING]   %add = add nsw i32 10, 5
--- Successfully hoisted 1 instructions. ---
```

Microbenchmark 3: tests\loop-invariant-code-motion\test3.c

This checks whether the transitive invariant instructions are captured by the pass. It confirms that our iterative approach works correctly.

```
opt -load-pass-plugin=build/unifiedpass.so -passes='loop-invariant-code-motion' build/t
2r.bc -o build/tests/loop-invariant-code-motion/test3-opt.bc

--- Analyzing Rotated Loop: for.body ---
    [SAFE - MARKED FOR HOISTING]   %add = add nsw i32 %a, %b
    [SAFE - MARKED FOR HOISTING]   %mul = mul nsw i32 %add, 5
    [SAFE - MARKED FOR HOISTING]   %add1 = add nsw i32 %mul, 10
--- Successfully hoisted 3 instructions. ---
```

Two simple benchmarks

- **Microbenchmark 4: tests\loop-invariant-code-motion\test4.c**
- **Microbenchmark 5: tests\loop-invariant-code-motion\test5.c**

Two negative benchmarks

- **Microbenchmark 6: tests\loop-invariant-code-motion\test6.c**
  - Instruction is loop depended. Cannot mark as invariant
  - Instruction not marked as invariant: CORRECT
  -
    ```
    opt -load-pass-plugin=build/unifiedpass.so -passes='loop-invarian
    2r.bc -o build/tests/loop-invariant-code-motion/test6-opt.bc

    --- Analyzing Rotated Loop: for.body ---
    ```

- **Microbenchmark 7: tests\loop-invariant-code-motion\test7.c**
  - Not dominating all exits. Cannot hoist the invariant instruction
  -
    ```
    opt -load-pass-plugin=build/unifiedpass.so -passes='loop-invariant-code-m
    2r.bc -o build/tests/loop-invariant-code-motion/test7-opt.bc

    --- Analyzing Rotated Loop: for.body ---
        [UNSAFE] Does not dominate all exits:   %mul = mul nsw i32 %x, %y
    ```

Dynamic instruction count reduction:

```
root@a39b751d1f03:/work# ./lli-compare.sh ./build/tests/loop-invariant-code-motion/
=====================================================================
Bitcode Pair          | m2r        | opt        | Diff (%)
=====================================================================
test1                 | 607        | 509        |    98 (+16.14%)
test2                 | 709        | 561        |   148 (+20.87%)
test3                 | 1200008    | 900012     | 299996 (+25.00%)
test4                 | 9000007    | 7000010    | 1999997 (+22.22%)
test5                 | 80807      | 71008      |  9799 (+12.13%)
test6                 | 7000007    | 7000008    |              -1
test7                 | 8000007    | 8000008    |              -1
=====================================================================
```

For negative testcases one extra instruction is added due to the loop rotation and insertion of the landing pad. All other test cases show that the LICM pass is able to optimize the programs heavily.

EXTRA CREDIT SECTION:

- Added Aggressive-LICM to relax the rules of the LICM.
- The LICMSafePass doesn't consider instructions with I.mayReadFromMemory() for LICM
- We have relaxed that condition.
- Now the LICM will hoist loads if the loop don't contain any other instructions that can modify the memory.
- This can result in higher optimizations for code that uses loads heavily without using stores in their loops

Analysis

```
root@a39b751d1f03:/work# ./lli-compare.sh ./build/tests/loop-invariant-code
=====================================================================
Bitcode Pair          | m2r        | opt        | Diff (%)
=====================================================================
agg                   | 9178       | 9180       |              -2
test1                 | 607        | 509        |    98 (+16.14%)
test2                 | 709        | 561        |   148 (+20.87%)
test3                 | 1200008    | 900012     | 299996 (+25.00%)
test4                 | 9000007    | 7000010    | 1999997 (+22.22%)
test5                 | 80807      | 71008      |  9799 (+12.13%)
test6                 | 7000007    | 7000008    |              -1
test7                 | 8000007    | 8000008    |              -1
=====================================================================
root@a39b751d1f03:/work# ./lli-compare.sh ./build/tests/aggressive-licm/
=====================================================================
Bitcode Pair          | m2r        | opt        | Diff (%)
=====================================================================
test1                 | 9178       | 6166       |  3012 (+32.82%)
=====================================================================
```

- agg.c and test1.c are the same test under 2 separate passes.
- A clear optimization can be observed under aggressive-licm